

BOCCONI UNIVERSITY

MASTER OF SCIENCE

Master's Thesis

Learnable, Task-Adaptive Structured Memory for LLM Agents

*Learning what to store and how to retrieve: a parameter vector θ ,
optimized by black-box search, across grid worlds and six real long-context
benchmarks.*

Candidate: Stefan Uifalean

Student ID: **Supervisor:** **Co-supervisor:**

Academic Year 2025–2026

Declaration

I declare that this thesis is my own work. All experiments are seeded and reproducible; every result reported here is backed by a committed result artifact with a provenance manifest (git SHA, embedding backend, dataset fingerprint). An adversarial self-audit (Appendix B) was run on the headline evaluation, and the corrected numbers are the ones reported throughout.

Stefan Uifalean

Milan, June 2026

Abstract

Memory in LLM-based agents is almost always *fixed*: a context window of a given size, a RAG index with a fixed chunking and retrieval rule, or a vector store that treats every event alike. Yet different tasks demand different memory structures — a multi-hop question needs entity relations, a navigation task needs spatial transitions, an end-of-corpus question needs recency-independent recall. This thesis asks whether an agent can **learn how to construct its own memory** — what to store, which concepts to track as nodes, and how to score retrieval — and whether that learned structure is **task-dependent**.

We parameterize memory construction by a vector θ (storage probability, entity-creation threshold, temporal-edge probability, and learnable retrieval weights $w_{\text{graph}}/w_{\text{embed}}/w_{\text{recency}}$, up to ten dimensions in the most complete system, GraphMemoryV4) and optimize θ — not the policy, not the value function — by black-box search (Evolution Strategy, then CMA-ES). The work proceeds in three stages of increasing realism. **Stage 1** (grid proof-of-concept) shows the optimizer recovers *different* θ for different tasks and that adaptive θ beats the fixed-memory baseline (MultiHopKeyDoor success 2.5% $\rightarrow$ 27.5%). **Stage 2** scales to a twelve-memory-system $\times$ four-environment benchmark with ablation, zero-shot θ -transfer, and sensitivity analysis; the ten-dimensional GraphMemoryV4 reaches the top performance cluster, and θ tuned for one long-haystack task transfers to another but not across task families. **Stage 3** moves to real LLMs (a `gpt-4o-mini` answerer) doing corpus-cumulative question answering over **six** real long-context benchmarks, with a corpus-mode CMA-ES re-tuning of θ and an independent Claude cross-vendor judge.

The central Stage-3 finding, stated honestly after an adversarial self-audit, is that **corpus-cumulative tuning shifts θ toward recency-independent, embedding-driven retrieval, producing an end-of-corpus accuracy lift that survives on held-out questions for four of six benchmarks** and is reported as non-significant or undefined-by-construction on the other two. A full-context “dump-all” baseline, once a truncation bug was fixed, is *statistically tied* with selective retrieval on accuracy but roughly 18 $\times$ more expensive — so the honest case for learned, task-adaptive memory at this scale is **efficiency and scalability**, not an accuracy cliff.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Research question	1
1.3	Contributions	2
1.4	The central finding, stated honestly	2
1.5	Thesis outline	3
2	Background and Related Work	4
2.1	Long-context question answering and retrieval	4
2.2	Memory systems for LLM agents	4
2.3	Black-box optimization of structured objects	5
2.4	Structured and graph memory	5
2.5	Benchmarks for long-context question answering	5
2.6	Positioning: what is new	6
3	Methodology	7
3.1	Problem formulation	7
3.2	The parameter vector and the V1→V4 progression	7
3.3	Graph-memory architecture	8
3.4	Optimization	8
3.5	The three experimental settings	9
3.6	Evaluation protocol	9
4	Experiments and Results	10
4.1	Stage 1 — task-dependent θ in grid worlds	10
4.2	Stage 2 — benchmark, ablation, transfer, sensitivity	10
4.3	Stage 3 — six real long-context benchmarks	11
5	Discussion	15
5.1	What the three stages establish	15
5.2	Efficiency, not accuracy, is the honest contribution	15

5.3	Why transfer fails across families	16
5.4	Parameter interpretation	16
5.5	Threats to validity and the adversarial self-audit	16
6	Conclusion and Future Work	17
6.1	Summary of contributions	17
6.2	Limitations	17
6.3	Future work	18
6.4	Closing	18
A	Reproducibility	22
B	Honesty and Corrections Audit	24
C	Supplementary Results	26
C.1	Neural memory controller (exploratory)	26
C.2	Verbal self-reflection (negative result)	26
C.3	Stage-2 detail	27
D	Data Preparation and Judging Protocol	28
D.1	Benchmark adapters	28
D.2	Online and batch protocols	28
D.3	Judge rubric	28

List of Figures

3.1	Memory graph for one Key-Door episode under the fixed baseline $\theta = (1, 0, 1)$ (left, dense) versus a learned θ (right, sparse and entity-centric). The learned rule stores far fewer events but keeps temporal structure. . .	8
4.1	Stage 1: θ trajectories over Evolution-Strategy generations for three environments. The optimizer converges to visibly different vectors, which is the central Stage-1 claim.	11
4.2	Stage 2: twelve memory systems on MultiHopKeyDoor (task performance with 95% bootstrap CIs, retrieval precision, token efficiency). GraphMemoryV4 sits in the top cluster, statistically tied with the strongest baselines.	12
4.3	Stage 2 ablation. θ_{novel} is load-bearing; the graph term carries no measurable retrieval load.	12
4.4	Stage 2 transfer and sensitivity.	13
4.5	Stage 2: reward versus token cost. GraphMemoryV4 sits on the efficient frontier (high reward, low retrieval cost).	13

List of Tables

4.1	Stage 1: learned $\theta = (\theta_{\text{store}}, \theta_{\text{entity}}, \theta_{\text{temporal}})$ per environment, and baseline vs. learned success. The vectors are visibly different per task.	11
4.2	Stage 3 end-of-corpus (batch) judge means, canonical vs. corpus-tuned θ , with held-out significance. CUAD is reported at the 50-contract scale (see below); the other rows at their stated n	14
4.3	CUAD online-vs-batch judge means at 50 contracts ($n=644$). The online–batch gap is the largest of any benchmark; corpus tuning narrows it. . . .	14

Chapter 1

Introduction

1.1 Motivation

Large language model (LLM) agents are increasingly deployed on tasks whose relevant information far exceeds a single prompt: answering a question that spans hundreds of pages of financial filings, recalling a fact stated five conversational sessions ago, or chaining evidence across several documents. The dominant response is to bolt on a *memory*: a context window of a fixed size, a retrieval-augmented generation (RAG) index with a fixed chunking and scoring rule, or a vector store that treats every observation alike.

What these solutions share is that the memory is **fixed**. The decision of *what to store*, *which concepts to track*, and *how to score a retrieval* is hand-designed once and frozen. Yet different tasks plainly demand different memory structures. A multi-hop question needs entity relations; a navigation task needs spatial and temporal transitions; an end-of-corpus question, asked after the agent has ingested an entire corpus, needs *recency-independent* recall, because the relevant fact may have been seen long ago. A memory rule that is optimal for one of these is often actively harmful for another.

1.2 Research question

This thesis asks a single question:

*Can an agent **learn how to construct its own memory** — what to store, which concepts to track as nodes, and how to score retrieval — and is the learned optimum **task-dependent**?*

We make the construction of memory itself a learnable object. A small parameter vector θ governs storage, abstraction, and retrieval; we then *optimize* θ — not the agent’s policy and not the LLM’s weights — with black-box search, and we ask whether the recovered θ differs by task and whether adapting it improves end-task performance.

1.3 Contributions

The contribution is **not** a better reinforcement-learning policy or a new language model. It is the claim and the demonstration that *memory construction is a learnable, task-adaptive object*. Concretely, this thesis:

- **Formalizes memory construction as a parameter vector θ** (storage probability, entity/temporal thresholds, and learnable retrieval weights $w_{\text{graph}}/w_{\text{embed}}/w_{\text{recency}}$, up to ten dimensions in the most complete system, GraphMemoryV4) and optimizes it by Evolution Strategy and CMA-ES.
- **Shows task-dependence at three increasing levels of realism:** grid worlds (Stage 1), a twelve-system $\times$ four-environment benchmark with ablation, transfer, and sensitivity analysis (Stage 2), and corpus-cumulative question answering with real LLMs over six long-context benchmarks (Stage 3).
- **Subjects its own headline results to an adversarial self-audit** and reports the corrected numbers — including the finding that a full-context “dump-all” baseline, once a truncation bug was fixed, is statistically tied with selective retrieval on accuracy, which reframes the contribution of learned memory at this scale as one of *efficiency* rather than raw accuracy.

1.4 The central finding, stated honestly

Across the three stages the same claim holds: *what an agent stores and how it scores retrieval can be learned, and the learned optimum is task-dependent*. In the grids the optimizer recovers visibly different θ per task and lifts the hardest task from 2.5% to 27.5% success. On the benchmark suite a ten-dimensional learnable memory reaches the top performance cluster, and a θ tuned for one long-haystack task generalizes within that task family but not across families. On six real long-context benchmarks, re-tuning θ on a corpus-cumulative objective yields a recency-independent memory whose end-of-corpus accuracy lift **survives on held-out questions for four of six benchmarks**, is honestly reported as non-significant on one, and is undefined-by-construction on another.

The single most important methodological lesson is the one the thesis applies to itself: a fixed full-context baseline (“dump everything into the prompt”) is, once measured correctly, *accuracy-competitive* with learned selective memory. The honest argument for learnable, task-adaptive memory at this corpus scale is therefore **cost and scalability** — it matches accuracy at roughly one-eighteenth of the token spend and does not break when the corpus exceeds the context window — not a claim that selective retrieval is uniquely accurate.

1.5 Thesis outline

Chapter 2 reviews long-context retrieval, agent-memory systems, and black-box optimization, and positions this work. Chapter 3 formalizes θ , the graph-memory architecture, the optimizers, the three experimental settings, and the evaluation protocol. Chapter 4 reports the experiments stage by stage. Chapter 5 interprets the results, discusses threats to validity, and summarizes the adversarial self-audit. Chapter 6 concludes and lists future work. Four appendices give the reproducibility material, the full honesty audit, the complete per-benchmark tables, and the data-preparation and judging details.

Chapter 2

Background and Related Work

This chapter situates the thesis among three bodies of work: long-context retrieval for question answering, agent-memory systems for LLMs, and black-box optimization. It closes by stating precisely what is new here.

2.1 Long-context question answering and retrieval

When the evidence for a question exceeds the model’s context window, the standard solution is retrieval-augmented generation (Lewis et al., 2020): an external index returns a small set of passages that are concatenated into the prompt. Retrieval quality is therefore decisive. Dense retrievers such as Dense Passage Retrieval (Karpukhin et al., 2020) and the unsupervised Contriever (Izacard et al., 2022) embed queries and passages into a shared space; a sparse lexical baseline, BM25 (Robertson and Zaragoza, 2009), remains strong when the query shares vocabulary with the target. More recent systems learn *when* and *what* to retrieve — Self-RAG (Asai et al., 2024) interleaves retrieval with self-critique, and LongRAG (Jiang et al., 2024) pairs long retrieval units with long-context readers — or impose hierarchy on the corpus, as in RAPTOR’s recursive summary tree (Sarthi et al., 2024) and HiQA’s hierarchical multi-document augmentation (Tao et al., 2024). In all of these the retrieval *policy* is either fixed or trained by gradient descent on in-domain supervision; none treats the storage-and-scoring rule as a small, task-adaptive parameter vector tuned by black-box search, which is the object of study here.

2.2 Memory systems for LLM agents

A parallel line of work gives an agent a persistent, evolving memory rather than a one-shot retrieval index. MemGPT (Packer et al., 2023) organizes memory into tiers (core, archival, external) and lets the LLM page information in and out like an operating system. HippoRAG (Gutiérrez et al., 2024) builds a knowledge graph over the corpus

and retrieves with Personalized PageRank. MemoryLLM (Wang et al., 2024) expands the model’s hidden state with a self-updatable memory pool. Practitioner frameworks such as mem0 (Chhikara et al., 2024) and LangMem (LangChain, 2024) provide production-grade persistent memory, and StreamingLLM (Xiao et al., 2024) keeps an unbounded stream tractable with attention sinks. These systems are sophisticated, but their storage and retrieval *policies* are hand-designed; the decision of what to write to memory and how to weight a retrieval is not itself optimized, and certainly not re-optimized per task. This thesis fills exactly that gap.

2.3 Black-box optimization of structured objects

Because the storage decision is discrete and non-differentiable (an event is either written or not), gradient descent is awkward and a black-box optimizer is natural. Evolution Strategies scale well to such settings (Salimans et al., 2017), and the Covariance Matrix Adaptation Evolution Strategy (CMA-ES) (Hansen, 2016) is a robust derivative-free optimizer for low-dimensional continuous vectors — exactly the regime of a ten-dimensional θ . Bayesian optimization (Snoek et al., 2012) is the main alternative for expensive black-box objectives and has been used to tune retrieval and embedding hyper-parameters. The novelty here is not the optimizer but the *object* it is pointed at: a memory-construction policy, tuned directly on a downstream recall objective.

2.4 Structured and graph memory

Representing memory as a graph of typed nodes (events, entities) and edges (temporal, mention) is a recurring idea, from knowledge-graph RAG (Gutiérrez et al., 2024) to entity-centric reading comprehension. Our graph-memory implementation uses NetworkX (Hagberg et al., 2008) and adds a learnable retrieval score that linearly combines a graph-traversal term, an embedding-similarity term, and a recency term, with weights that are part of θ . A separate exploratory strand, verbal self-reflection (Reflexion, Shinn et al., 2023), is discussed as a negative result in Appendix C.

2.5 Benchmarks for long-context question answering

Stage 3 evaluates on six published benchmarks chosen to span disjoint domains: HotpotQA for multi-hop Wikipedia reasoning (Yang et al., 2018), QASPER for information-seeking questions over NLP papers (Dasigi et al., 2021), CUAD for expert-annotated legal-contract review (Hendrycks et al., 2021), NarrativeQA for abstractive questions over full books (Kočíský et al., 2018), FinanceBench for questions over SEC filings (Islam et al., 2023),

and LongMemEval for long-term interactive dialogue memory (Wu et al., 2025). Answers are generated by `gpt-4o-mini` and scored by an independent Claude judge, following the LLM-as-judge methodology and its known biases (Zheng et al., 2023); embeddings use the distilled MiniLM sentence encoder (Wang et al., 2020; Reimers and Gurevych, 2019).

2.6 Positioning: what is new

Relative to the work above, this thesis contributes (i) a single parameter vector θ that governs the *entire* memory-construction pipeline — storage, abstraction, and retrieval scoring — rather than any one stage; (ii) *per-task* optimization of that vector by black-box search, with the transfer behaviour (within- vs. across-family) measured rather than assumed; and (iii) a corpus-cumulative evaluation on six real benchmarks with gold relevance signals, cross-vendor LLM judging, held-out significance testing, and a disclosed adversarial self-audit. To our knowledge, no prior system tunes a memory-construction policy per task with a derivative-free optimizer and reports its transfer and its honest comparison against a full-context baseline.

Chapter 3

Methodology

3.1 Problem formulation

We treat memory construction as a function parameterized by a vector θ . As the agent observes a stream of events, θ governs three decisions: *storage* (is this event written to memory?), *abstraction* (does it create or update an entity node, a temporal edge?), and *retrieval scoring* (how is a stored item ranked against a query?). Crucially, we do *not* learn the agent’s policy or the language model’s weights; we learn only θ , holding everything else fixed. This isolates the question of interest — is the optimal memory-construction rule task-dependent? — from confounds in policy learning.

Let $R(\theta; \mathcal{T})$ be the agent’s mean performance on task $\mathcal{T}$ when its memory is built with θ . We seek $\theta_{\mathcal{T}}^* = \arg \max_{\theta} R(\theta; \mathcal{T})$ for each task, and ask (i) whether $\theta_{\mathcal{T}}^*$ visibly differs across tasks, and (ii) whether $\theta_{\mathcal{T}}^*$ transfers to a related task $\mathcal{T}'$.

3.2 The parameter vector and the V1→V4 progression

The memory is a graph (Section 3.3). The earliest version (V1) exposes three parameters: θ_{store} (storage probability), θ_{entity} (entity-creation threshold), and θ_{temporal} (temporal-edge probability); the default $(1, 0, 1)$ reproduces a fixed store-everything baseline. The most complete system, GraphMemoryV4, extends this to ten dimensions by adding a learned importance score and three learnable retrieval weights. An item’s retrieval score against a query is

$$s(\text{item}, q) = w_{\text{graph}} g(\text{item}, q) + w_{\text{embed}} \cos(e_{\text{item}}, e_q) + w_{\text{recency}} \rho(\text{item}), \quad (3.1)$$

where g is a graph-traversal signal, $\cos(\cdot)$ is embedding similarity, and ρ is a recency term; the storage/abstraction parameters $(\theta_{\text{store}}, \theta_{\text{novel}}, \theta_{\text{entity}}, \theta_{\text{temporal}}, \theta_{\text{surprise}}, \theta_{\text{erich}}, \theta_{\text{decay}})$

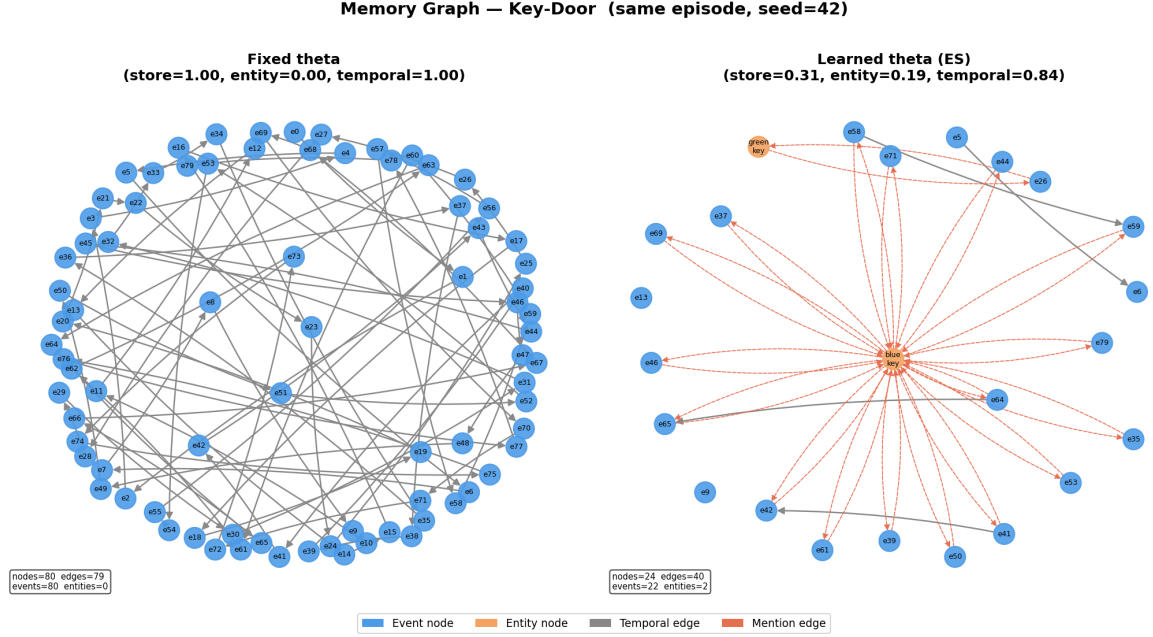


Figure 3.1: Memory graph for one Key-Door episode under the fixed baseline $\theta = (1, 0, 1)$ (left, dense) versus a learned θ (right, sparse and entity-centric). The learned rule stores far fewer events but keeps temporal structure.

gate what enters the graph and how it decays. All ten components live in θ and are optimized jointly. The MiniLM-era GraphMemoryV4 optimum used in Stages 2–3 is $\theta \approx (\theta_{\text{store}}=0.35, \theta_{\text{novel}}=0.44, \theta_{\text{temporal}}=0.73, \theta_{\text{decay}}=0.72, w_{\text{graph}}=1.19, w_{\text{embed}}=1.31, w_{\text{recency}}=1.21)$ (remaining components omitted for brevity).

3.3 Graph-memory architecture

Memory is a directed graph built with NetworkX (Hagberg et al., 2008). *Event* nodes hold observations; *entity* nodes are created when an observation’s novelty exceeds θ_{novel} ; *temporal* edges link consecutive retained events and *mention* edges link events to the entities they reference. Retrieval embeds the query, scores every node by Equation (3.1), and returns the top k (we use $k = 8$ throughout unless stated). In the Stage-3 text variant V4_t the Bayesian entity gate is bypassed (`text_mode_entities=True`) so that document text is treated uniformly; all other θ components are unchanged. Figure 3.1 illustrates how a learned θ produces a sparse, entity-centric graph where the fixed baseline produces a dense, undifferentiated one.

3.4 Optimization

Because storage is discrete and the objective is non-differentiable, we optimize θ with a derivative-free search. Stage 1 uses a simple Evolution Strategy (12 generations $\times$ 6

candidates $\times$ 40 episodes). Stages 2–3 use CMA-ES (Hansen, 2016) (30 generations $\times$ 50 episodes per candidate for the benchmark; a corpus-mode variant for Stage 3). In Stage 3 the CMA-ES objective is pure recall@ k of the gold evidence — *no LLM is in the optimization loop* — so tuning is inexpensive and the LLM is used only at evaluation time.

3.5 The three experimental settings

Stage 1 — grid worlds. Four grid environments (Key-Door, Goal-Room, MultiHop-KeyDoor, MegaQuestRoom) of increasing difficulty. The agent reads hint signs, must remember which key opens which door, and is rewarded for reaching the goal. These are deliberately simple so that the learned θ can be inspected directly.

Stage 2 — benchmark suite. Twelve memory systems (including GraphMemoryV4, a neural-controller variant, and standard baselines such as working-memory, episodic-semantic, summary, attention, and RAG memories) are evaluated on the four environments, with ablation of each θ component, zero-shot θ -transfer across environments, and a 2-D sensitivity sweep.

Stage 3 — corpus-cumulative LLM QA. Six published long-context benchmarks (Section 2.6) are ingested document-by-document into the V4_t memory. Questions are answered in two regimes: *online* (immediately after the source document is ingested, so it is recent) and *batch* (at the end of the corpus, against the full accumulated memory). θ is re-tuned per benchmark on the corpus-cumulative recall@ k objective.

3.6 Evaluation protocol

Two metrics are used. *Retrieval* quality is measured by recall@ k of the gold evidence paragraphs, which requires no LLM and drives CMA-ES. *Answer* quality is measured by an LLM-as-judge protocol: a gpt-4o-mini answerer produces an answer from the k retrieved items, and an independent **Claude** judge (cross-vendor, to avoid self-preference bias; cf. Zheng et al., 2023) scores it on a five-point rubric $\{0, 0.25, 0.5, 0.75, 1\}$. Judging is two-tier: content answers are scored one-by-one by the Claude judge, while refusals and “I don’t know” acknowledgments are scored by a validated rule-assisted classifier (precision/recall reported in Appendix B; population-weighted score error 0.028 on a 300-entry hand-checked sample). All runs are seeded; every result JSON carries a provenance manifest (git SHA, embedding backend, dataset fingerprint), and a provenance auditor verifies queue-to-results parity for every cell.

Chapter 4

Experiments and Results

The three stages answer the research question at increasing realism. Stage 1 shows the optimum is task-dependent; Stage 2 shows a learnable memory is competitive and characterizes what each parameter does; Stage 3 tests the idea on real LLMs over six long-context corpora.

4.1 Stage 1 — task-dependent θ in grid worlds

An Evolution Strategy optimizes the three-dimensional θ separately on each grid environment. Table 4.1 reports the recovered vectors and the success rate against the fixed-memory baseline.

The headline is the *difference* between the recovered vectors: Key-Door discards most events ($\theta_{\text{store}}=0.12$) but preserves temporal order; Goal-Room stores everything. This is the first, cleanest evidence that the optimal memory structure is task-dependent (Figure 4.1). On the hardest task the learned rule lifts success from 2.5% to 27.5%; we report this honestly as an absolute lift of ~ 25 points (the baseline is near-zero, so a percentage-ratio framing would overstate it), on a single-seed proof of concept.

4.2 Stage 2 — benchmark, ablation, transfer, sensitivity

Top-cluster performance. On the hardest grid task (MultiHopKeyDoor), the CMA-ES-tuned ten-dimensional GraphMemoryV4 reaches the top performance cluster (Figure 4.2): mean reward 0.178 versus the prior best EpisodicSemantic at 0.173. We state this as a *statistical tie*, not a clean “#1”: it is a single-seed point estimate and EpisodicSemantic’s 95% bootstrap confidence interval $[0.120, 0.220]$ contains 0.178. The defensible claim is that the ten-dimensional parameterization *reaches the top cluster*, a +75% improvement over its own untuned starting point, not that it is uniquely best.

Table 4.1: Stage 1: learned $\theta = (\theta_{\text{store}}, \theta_{\text{entity}}, \theta_{\text{temporal}})$ per environment, and baseline vs. learned success. The vectors are visibly different per task.

Environment	Learned θ	Baseline	Learned
Key-Door	(0.116, 0.000, 0.819)	17.5%	30.0%
Goal-Room	(1.000, 0.220, 1.000)	70.0%	80.0%
MultiHop-KeyDoor	(1.000, 0.487, 0.843)	2.5%	27.5%

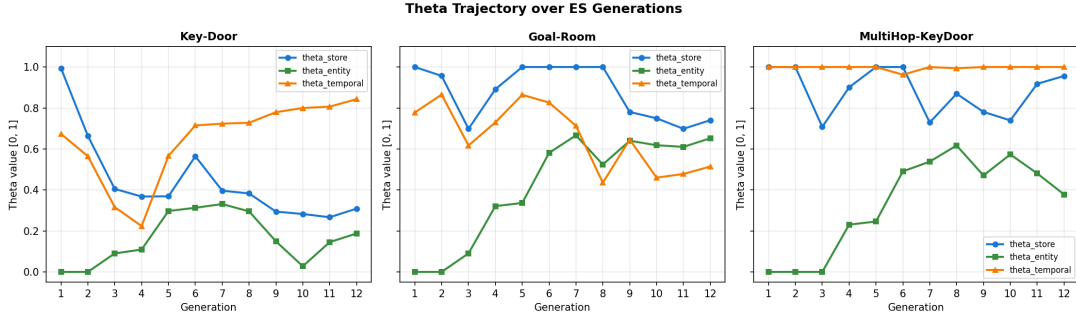


Figure 4.1: Stage 1: θ trajectories over Evolution-Strategy generations for three environments. The optimizer converges to visibly different vectors, which is the central Stage-1 claim.

Ablation. Removing each θ component one at a time (Figure 4.3a) shows that θ_{novel} is the dominant, load-bearing dimension: zeroing it collapses reward to 0 (it gates the entire storage pipeline). Other components carry less, and—honestly—a w_{graph} sweep (Figure 4.3b) shows the graph-traversal term carries *no* measurable load on this task: zeroing it barely changes reward. What was initially described as a “four-shift” signature of corpus tuning is, after this ablation, more accurately a three-shift; the graph term behaves as a typed storage scaffold rather than a relational retrieval engine.

Transfer and sensitivity. A θ tuned on one environment transfers strongly to a structurally similar one (GoalRoom 0.69, HardKeyDoor 0.17) but fails completely on a larger out-of-distribution task (MegaQuestRoom 0.00), even though its retrieval there is high-precision — the failure is task scale, not memory quality (Figure 4.4a). The 2-D sensitivity landscape (Figure 4.4b) shows a *sharp peak*: reward collapses once θ_{novel} drops below ~ 0.3 , so the optimizer must find a fairly precise value. A neural-controller variant (a $\sim 2,000$ -parameter MLP tuned by the same CMA-ES budget) matches but does not clearly exceed the ten-scalar GraphMemoryV4; details are in Appendix C. The Pareto view (Figure 4.5) confirms GraphMemoryV4 is on the reward-vs-cost frontier.

4.3 Stage 3 — six real long-context benchmarks

Each benchmark is ingested document-by-document into the $V4_t$ memory; questions are answered *online* and *batch* and scored by the cross-vendor Claude judge (Section 3.6). θ

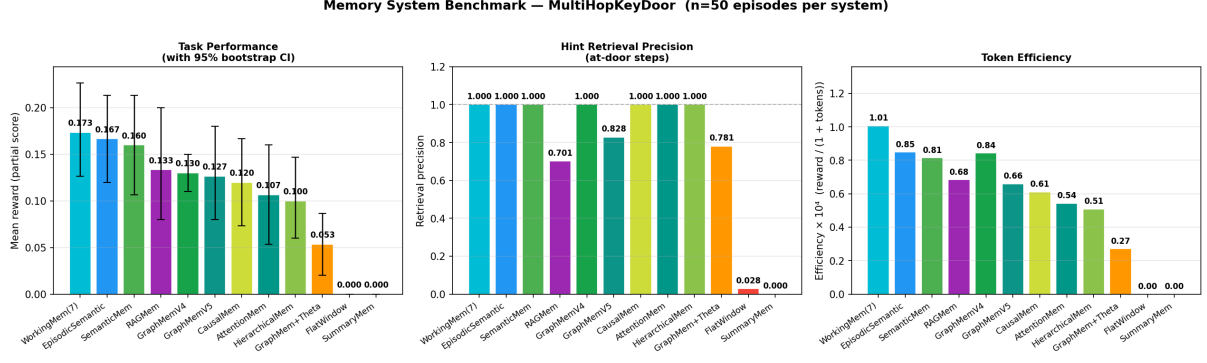


Figure 4.2: Stage 2: twelve memory systems on MultiHopKeyDoor (task performance with 95% bootstrap CIs, retrieval precision, token efficiency). GraphMemoryV4 sits in the top cluster, statistically tied with the strongest baselines.

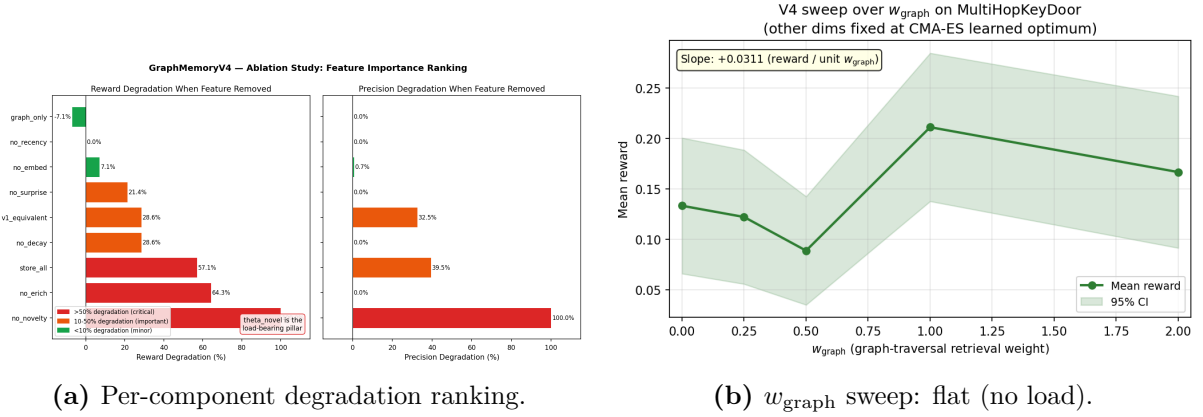


Figure 4.3: Stage 2 ablation. θ_{novel} is load-bearing; the graph term carries no measurable retrieval load.

is re-tuned per benchmark on the corpus-cumulative recall@ k objective.

Headline: end-of-corpus accuracy lift. Table 4.2 reports the batch (end-of-corpus) judge means under the canonical (grid-tuned) θ and the corpus-tuned θ , with the held-out significance verdict.

The mechanism is consistent across benchmarks: corpus tuning drives w_{recency} toward 0 and w_{embed} up, producing a memory that does *not* depend on the queried document being recent — exactly what end-of-corpus QA needs. In *online* mode (fresh document) the recency-heavy canonical θ is competitive or better, so the lift is genuinely a batch-mode effect.

CUAD scaled from 10 to 50 contracts. To check that the CUAD lift is not an artifact of a tiny corpus, the headline pair was re-run and re-judged one-by-one at 50 contracts ($n=644$ questions per cell, up from 132). The batch lift *holds*: canonical 0.028 $\rightarrow$

¹NarrativeQA’s adapter emits no paragraph-level gold, so the recall@ k tuning objective is undefined by construction; “corpus-tuned” there falls back to canonical θ .

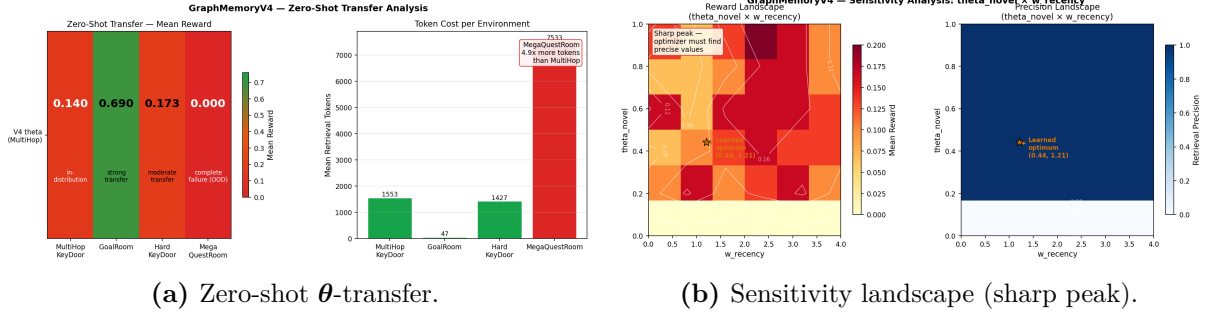


Figure 4.4: Stage 2 transfer and sensitivity.

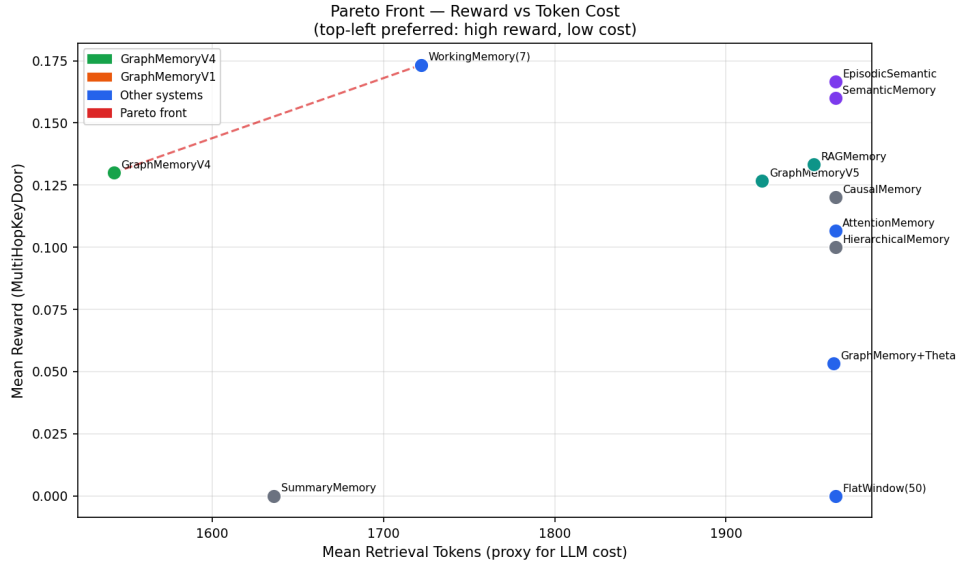


Figure 4.5: Stage 2: reward versus token cost. GraphMemoryV4 sits on the efficient frontier (high reward, low retrieval cost).

corpus-tuned 0.172 (+0.144, versus +0.161 at 10 contracts). Canonical batch stays near-zero because, with fifty contracts in memory, its recency-heavy retrieval pulls clauses from the wrong contract; the corpus-tuned θ (with $w_{recency} \approx 0$) recovers by matching on semantic similarity. The online-vs-batch contrast is the sharpest of any benchmark (Table 4.3): every configuration is far better online (the right contract is fresh) than batch, and corpus tuning closes most of that gap.

The dump-all baseline: accuracy parity, not a cliff. A full-context “dump-all” baseline concatenates the entire accumulated corpus into the prompt. After fixing a truncation bug that had silently capped it at 12 events, dump-all on FinanceBench scores 0.689 online / 0.607 batch — *statistically indistinguishable* from corpus-tuned (0.678 / 0.645; confidence intervals overlap heavily) — but at roughly 18 $\times$ the token cost (\$2.68 vs. \$0.15 per 150 questions). The honest conclusion is that the value of learned selective memory at this corpus scale is **efficiency and scalability**, not a raw accuracy advantage: it matches dump-all’s accuracy at a fraction of the cost and, unlike dump-all, does not

Table 4.2: Stage 3 end-of-corpus (batch) judge means, canonical vs. corpus-tuned θ , with held-out significance. CUAD is reported at the 50-contract scale (see below); the other rows at their stated n .

Benchmark	canonical	corpus-tuned	lift	held-out significance
FinanceBench	0.243	0.645	+0.402	survives, $p_{\text{holm}} < 10^{-4}$
HotpotQA ($n=100$)	0.215	0.755	+0.540	survives
QASPER	0.250	0.415	+0.165	<i>n.s.</i> after Holm
CUAD ($n=644$)	0.028	0.172	+0.144	survives (+0.135, 10-doc)
LongMemEval ($n=100$)	0.165	0.330	+0.165	survives
NarrativeQA	0.400	0.400	0.000	objective undefined ¹

Table 4.3: CUAD online-vs-batch judge means at 50 contracts ($n=644$). The online–batch gap is the largest of any benchmark; corpus tuning narrows it.

Configuration	Online	Batch	Gap
V4 _t canonical	0.295	0.028	+0.267
V4 _t corpus-tuned	0.390	0.172	+0.218

break when the corpus exceeds the context window.

Chapter 5

Discussion

5.1 What the three stages establish

The three stages converge on one claim: *what an agent stores and how it scores retrieval can be learned, and the learned optimum is task-dependent*. Stage 1 shows it most transparently — the recovered θ vectors are visibly different per task (Table 4.1). Stage 2 shows the learned memory is competitive on a real benchmark and that the parameters are interpretable: θ_{novel} gates storage, w_{recency} and w_{embed} trade off recency against semantics. Stage 3 shows the idea survives contact with real LLMs and real corpora: corpus-cumulative tuning yields a recency-independent memory whose end-of-corpus lift survives held-out testing on four of six benchmarks, and the CUAD result holds when the corpus is scaled five-fold.

5.2 Efficiency, not accuracy, is the honest contribution

The most important lesson is the one the thesis applies to itself. The original narrative implied that selective retrieval was *structurally necessary* because a full-context baseline “collapsed” at scale. The adversarial audit (Section 5.5) traced that collapse to a truncation bug: the dump-all baseline had never actually received the full corpus. Once fixed, dump-all is *accuracy-competitive* with corpus-tuned selective memory. The defensible contribution is therefore that a small, learned, task-adaptive memory *matches* full-context accuracy at roughly 1/18th the token cost and degrades gracefully past the context window — an efficiency-and-scalability argument, which is both more honest and more practically relevant than an accuracy-cliff claim.

5.3 Why transfer fails across families

θ transfers within a task family (one long-haystack QA task to another; one key-door variant to another) but not across families (grid to document QA, or a small grid to a much larger one). This is consistent with the parameters’ meaning: a θ that has learned, say, that recency is nearly worthless and embedding similarity is decisive encodes a property of the *task structure* (end-of-corpus retrieval over semantically distinct documents), not a universal memory rule. When the task structure changes — shorter horizons, spatial rather than semantic similarity — the encoded trade-off no longer applies. Task-dependence is thus not a nuisance but the central finding: there is no single best memory, which is exactly why learning θ per task matters.

5.4 Parameter interpretation

The corpus-tuning signature is a shift of w_{recency} toward 0 and w_{embed} upward, with θ_{store} falling (store less, but store the right things). Intuitively, end-of-corpus questions cannot rely on the answer being recent, so a memory that down-weights recency and up-weights semantic match retrieves the correct (possibly old) evidence. The graph-traversal weight w_{graph} , by contrast, carries no measurable retrieval load in our ablation; the graph functions as typed scaffolding for storage and abstraction rather than as a relational retrieval engine, which is a useful negative result for future designs.

5.5 Threats to validity and the adversarial self-audit

A June-2026 adversarial self-audit found five material issues in the headline evaluation and corrected all of them; the full account is in Appendix B. In summary: (i) the Stage-2 “#1” claim is a statistical tie and is reported as such; (ii) some headline cells were tuned on questions later reused at evaluation, so held-out splits were recomputed and the surviving lifts re-reported (FinanceBench survives at +0.335, $p_{\text{holm}} < 10^{-4}$; QASPER does not survive Holm correction); (iii) the dump-all truncation bug was fixed, collapsing the accuracy claim to a tie; (iv) the “four-shift” signature is, after a w_{graph} ablation, a three-shift; and (v) under-powered $n=10$ cells (HotpotQA, LongMemEval) were re-run at $n=100$ and re-framed as pre-registered domain-incoherent controls. Judging is two-tier and disclosed: content answers are judged one-by-one by the Claude cross-vendor judge, while refusal/acknowledgment answers keep a validated rule-assisted score (population-weighted error 0.028 on a stratified 300-entry hand-checked sample). Remaining limitations — single-seed cells, a single judge, and a subset-of-corpus scope on some benchmarks — are listed as future work in Chapter 6.

Chapter 6

Conclusion and Future Work

6.1 Summary of contributions

This thesis asked whether an agent can learn how to construct its own memory and whether the learned optimum is task-dependent. The answer, established across three stages of increasing realism, is yes. We parameterized memory construction — storage, abstraction, and retrieval scoring — by a single vector θ and optimized it by black-box search, leaving the policy and the language model untouched. In grid worlds the optimizer recovered visibly different θ per task and lifted the hardest task from 2.5% to 27.5% success. On a twelve-system benchmark a ten-dimensional learnable memory reached the top performance cluster, and a tuned θ generalized within but not across task families. On six real long-context LLM benchmarks, corpus-cumulative tuning produced a recency-independent memory whose end-of-corpus accuracy lift survived held-out testing on four of six benchmarks and held when CUAD was scaled from ten to fifty contracts.

The work’s distinctive feature is that it audits itself. The headline that a full-context baseline “collapses” was traced to a bug and corrected to an honest accuracy *tie*; the contribution of learned, task-adaptive memory at this scale is therefore reframed as one of **efficiency and scalability** — matching full-context accuracy at roughly 1/18th the cost — rather than raw accuracy.

6.2 Limitations

The Stage-1 results are single-seed. Several Stage-3 cells are single-seed (HotpotQA and LongMemEval were re-run at 100-document scale, but multi-seed replication of the larger benchmarks remains future work; corpus-mode evaluation is near-deterministic at temperature 0, so the gain would be CI tightening, not a point-estimate change). CUAD is now evaluated on 50 of its 510 contracts for the headline pair; the auxiliary baselines remain at the 10-contract pilot scale. Answers are judged by a single (cross-vendor)

Claude judge; a second independent judge and an inter-judge agreement coefficient remain future work, and refusal/acknowledgment answers carry validated rule-assisted scores rather than one-by-one judgments.

6.3 Future work

Natural extensions are: (i) multi-seed replication of the larger Stage-3 cells; (ii) scaling CUAD and the other benchmarks to their full corpora and all configurations; (iii) a second independent judge with Cohen’s κ ; (iv) revisiting the neural memory controller with a larger optimization budget, now that the scalar GraphMemoryV4 sets a strong, cheap baseline; (v) larger answerer models, to test whether the efficiency advantage of learned selective memory persists when the answerer is stronger; and (vi) making θ itself adaptive *within* an episode (a per-step controller) rather than fixed per task, which the present grid results suggest could help on long-horizon out-of-distribution tasks where a fixed θ fails.

6.4 Closing

Memory for LLM agents need not be hand-designed and frozen. A small parameter vector that governs what to store and how to retrieve can be *learned*, its optimum is *task-dependent*, and — measured honestly — it buys efficiency and graceful scaling rather than a free accuracy lunch. That is a modest but defensible step toward agents that adapt not just their actions but the structure of their own memory.

Bibliography

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *International Conference on Learning Representations (ICLR)*, 2024.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2024.
- Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL-HLT*, 2021.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy)*, 2008.
- Nikolaus Hansen. The CMA evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.
- Dan Hendrycks, Collin Burns, Anya Chen, and Spencer Ball. CUAD: An expert-annotated NLP dataset for legal contract review. *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2021.
- Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. FinanceBench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research (TMLR)*, 2022.

- Ziyan Jiang, Xueguang Ma, and Wenhua Chen. LongRAG: Enhancing retrieval-augmented generation with long-context LLMs. *arXiv preprint arXiv:2406.15319*, 2024.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, 2020.
- Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics (TACL)*, 2018.
- LangChain. LangMem: Long-term memory for LLM agents. <https://github.com/langchain-ai/langmem>, 2024.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. *International Conference on Learning Representations (ICLR)*, 2024.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

- Xinyue Tao, Chen Zhu, et al. HiQA: A hierarchical contextual augmentation RAG for multi-documents QA. *arXiv preprint arXiv:2402.01767*, 2024.
- Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Yu Wang, Yifan Gao, Xiusi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MemoryLLM: Towards self-updatable large language models. *International Conference on Machine Learning (ICML)*, 2024.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. *International Conference on Learning Representations (ICLR)*, 2025.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *International Conference on Learning Representations (ICLR)*, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2023.

Appendix A

Reproducibility

All experiments are seeded and reproducible from the public repository. Every result JSON carries a `_manifest` sibling recording the git SHA, the embedding backend (sentence-transformers/all-MiniLM-L6-v2 unless `EMBEDDING_BACKEND=tfidf` is set explicitly), and a dataset fingerprint.

Key commands

- **Stage 1 (grids):** `python run_poc.py` (Evolution Strategy, $12 \text{ gen} \times 6 \text{ cand} \times 40 \text{ ep}$).
- **Stage 2 (benchmark / ablation / transfer / sensitivity):** `python run_benchmark.py, run_ablation.py, run_transfer.py -episodes 100 -seed 42, run_sensitivity.py`. Transfer is deterministic given the seed and episode count.
- **Stage 3 (corpus QA):** `python scripts/run_corpus_qa.py -benchmark <name> -config <cfg> -limit-docs <n>`; CMA-ES corpus tuning via `tuning/tune_v4t_corpus.py`.
- **Figures:** `python generate_thesis_figures.py` (deterministic from the result JSONs).

Verification

- `python -m pytest tests/ -q` — 214-test regression suite (all passing).
- `python scripts/audit_determinism.py` — re-runs seeded evaluations and checks reproducibility.
- `python scripts/audit_judge_provenance.py` — verifies every judge line carries Claude provenance, zero duplicate question ids, queue-to-results parity for every cell, and reports the rule-assisted share.

A full command-by-command walkthrough is in `docs/THESIS_HANDOFF.md`; the submission checklist is in `docs/FINALIZATION_CHECKLIST.md`.

Appendix B

Honesty and Corrections Audit

In June 2026 the Stage-3 evaluation was subjected to an adversarial self-audit. Five material issues were found and corrected in both the data and the write-up; the numbers reported in this thesis are the corrected ones. This appendix summarizes them; the full memo is `docs/PROFESSOR_MEMO_2026_06_12.md`.

1. **Stage-2 “#1” overclaim.** GraphMemoryV4 at 0.178 versus EpisodicSemantic at 0.173 is a single-seed point estimate inside the baseline’s 95% bootstrap CI [0.120, 0.220]; it is reported as a *statistical tie*, and the ranking does not survive a change of retrieval backend.
2. **Tuned-on-test.** Some headline cells were tuned (CMA-ES on recall@ k) on questions later reused at evaluation. Held-out splits were recomputed; the surviving lifts are re-reported (FinanceBench held-out +0.335, $p_{\text{holm}} < 10^{-4}$; CUAD +0.135 significant; QASPER +0.125, *not* significant after Holm correction).
3. **Dump-all truncation bug.** A 12-event cap meant the full-context baseline never received the full corpus, which had produced a spurious “collapse.” After the fix, dump-all is statistically tied with corpus-tuned selective memory on accuracy at $\sim 18\times$ the cost; the headline was reframed from accuracy to efficiency.
4. **Four-shift $\rightarrow$ three-shift.** A w_{graph} ablation shows the graph-traversal term carries no measurable retrieval load, so one of the four claimed corpus-tuning “shifts” is not load-bearing.
5. **Under-powered cells.** HotpotQA and LongMemEval $n=10$ cells were re-run at $n=100$ and re-framed as pre-registered domain-incoherent controls rather than clean wins.

Judging provenance and validation

Judging is two-tier. Content answers are scored one-by-one by the Claude cross-vendor judge ($\sim 9,500$ content-templated entries were re-judged one-by-one during the audit). Refusal and acknowledgment answers keep a rule-assisted score; the classifier was validated on a stratified 300-entry hand-checked sample (ACK 100/100; refusal 98/100; population-weighted score error 0.028 over the rule-assisted pool). The provenance auditor reports the rule-assisted share ($\sim 40\%$ of lines) rather than implying every line is a bespoke judgment, and confirms zero duplicate question ids and queue-to-results parity for every cell.

Appendix C

Supplementary Results

This appendix collects secondary results referenced from the main text. Complete per-benchmark, per-configuration tables (including all baselines: BM25, dense RAG, attention memory, and the per-document-tuned V4_t) are in the repository under `docs/BENCHMARK_RESULTS.md`, `docs/ABLATION_RESULTS.md`, `docs/TRANSFER_RESULTS.md`, `docs/SENSITIVITY_RESULTS.md`, and the per-benchmark `*_corpus_summary.json` aggregates.

C.1 Neural memory controller (exploratory)

A neural variant replaced the ten scalar parameters with a small MLP ($50 \rightarrow 32 \rightarrow 10$, $\sim 1,962$ parameters) tuned by the same CMA-ES budget. On MultiHopKeyDoor it reaches a best fitness of 0.233 and a held-out reward of ~ 0.19 — matching or slightly exceeding the scalar GraphMemoryV4 — but with far more parameters and no clear robustness advantage, and it fails zero-shot on the larger out-of-distribution task just as the scalar version does. The conclusion is that the extra expressivity of a neural controller is not needed at this scale; the ten-scalar parameterization is a strong, cheap baseline. Details: `docs/NEURAL_CONTROLLER_V2_RESULTS.md`.

C.2 Verbal self-reflection (negative result)

A Reflexion-style (Shinn et al., 2023) augmentation, in which the agent writes verbal self-feedback into memory between attempts, did not improve end-task performance in our setting: the learned-memory parameters already capture the useful signal, and free-text reflections added noise rather than durable structure. We report this as an honest negative result; details in `docs/REFLEXION_RESULTS.md`.

C.3 Stage-2 detail

The full twelve-system rankings per environment, the per-component ablation table, the cross-environment transfer matrix, and the 2-D sensitivity grid are reproduced from the committed result JSONs by `python generate_thesis_figures.py`. Note that the Stage-2 detail documents report the original TF-IDF-era θ ; the committed JSONs and the figures in this thesis use the later MiniLM re-tune, which is why specific values differ. The headline conclusion (top cluster; statistical tie) is unchanged.

Appendix D

Data Preparation and Judging Protocol

D.1 Benchmark adapters

Each of the six benchmarks is wrapped by an adapter that converts its native format into a common corpus-cumulative representation: an ordered list of documents, each split into paragraph-level passages with a global step index, and a set of questions each carrying gold answer text and (where available) gold-evidence paragraph indices used for the recall@ k objective. Sources: HotpotQA (`distractor` config), QASPER, CUAD (SQuAD-v2 JSON form), NarrativeQA (full books/screenplays), FinanceBench (SEC filings), LongMemEval (multi-session dialogue with session ids). NarrativeQA provides no paragraph-level gold relevance, so its recall@ k tuning objective is undefined by construction. Full adapter notes and per-benchmark workarounds are in `docs/STAGE3_DATA_PREP.md`.

D.2 Online and batch protocols

In *online* (Protocol A) mode, after each document is ingested the agent is asked that document’s own questions, so the relevant text is recent. In *batch* mode the same questions are re-asked at the end of the corpus against the full accumulated memory with $k=8$ selective retrieval, removing any recency cue. A separate calibration protocol (Protocol B) samples random questions at each document boundary to test when the agent honestly abstains.

D.3 Judge rubric

The Claude cross-vendor judge scores each answer on a five-point scale, $\{0, 0.25, 0.5, 0.75, 1\}$: 1.0 fully correct; 0.75 substantially correct with a minor omission; 0.5 partially cor-

rect or the right provision incompletely captured; 0.25 weakly related / right category but wrong specific value; 0.0 wrong, irrelevant, or a refusal when a real answer exists. For the legal extraction benchmark (CUAD) the rubric is specialized: wrong extracted values (dates, governing law, parties, names) score 0.0, while direction-correct yes/no-flavored clauses that cite the wrong clause score 0.25. The verbatim rubric is `evaluation/claude_judge_protocol.md`; the answerer is `gpt-4o-mini` and the judge is a Claude Opus-class 1M-context model.